

two activities require the same resource simultaneously. For example, consider the case displayed in Figure 4. Assume that, in this case, the machine has a single shared memory pipeline for data and instructions.

In the above example, let's suppose that *i0* is a load instruction, which reads from memory. As a result, when the instruction *i0* contains a data-memory reference (MA), it will conflict with the instruction reference for *i3* (IF). To resolve this, we stall the pipeline for one clock cycle when a data-memory access occurs. I will explain the concept of stalls later in this article.

Control hazard

A control hazard occurs when an instruction changing PC (such as a branch instruction) is processed in a pipeline. This type of hazard is caused by uncertainty of the execution path, and the branch taken or not taken. If a branch changes the PC to its target address, it is a 'taken branch'; if it falls through, it is not taken. If Instruction *i* is a taken branch, then the PC is normally not changed until the end of the MEM stage—after the completion of the address calculation and comparison.

For example, consider the case in Figure 5. Assume that *i0* is a branch instruction, and *i1* is the branch successor.

Since *i0* is a branch instruction, the new PC will be determined in its MA stage. Till that time, it is not decided what instruction should be fetched next (IF of *i1*). To deal with this situation, we often stall the pipeline as soon as the branch is detected, until we reach the MEM stage, which determines the new PC.

Thus, we have seen three types of pipeline hazards, which I have summarised in Figure 6. Now that we have seen what pipeline hazards are, and what can cause them, let's look at how we can deal with them.

Dealing with pipeline hazards

Pipeline hazards can be overcome by either of, or a combination of:

- Creating a stall
- Bypassing (data forwarding)

Creating stalls

We can overcome pipeline hazards by inserting a stall in the pipeline. A pipeline stall is a method for preventing data, structural, and control hazards from occurring. As instructions are fetched, control logic determines whether a hazard could/will occur. If yes, then the control logic inserts NOPs into the pipeline. Thus, before the next instruction (which would cause the hazard) is executed, the previous one will have had sufficient time to complete, and thus prevent the hazard. Let's look at the same example that we discussed for structure hazards—take another look at Figure 4.

Here, *i0* is a load instruction, which reads from memory. As a result, when the instruction *i0* contains a data-memory reference (MA), it will conflict with the instruction reference for *i3* (IF). To resolve this, we stall the pipeline for one clock cycle when a data-memory access occurs. That is, we delay the processing of *i3* by one clock cycle, to resolve the resource conflict with *i0*, as shown in Figure 7.

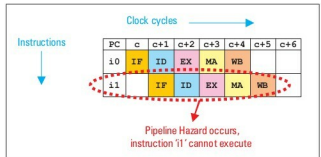


Figure 1: A pipeline hazard prevents instruction execution

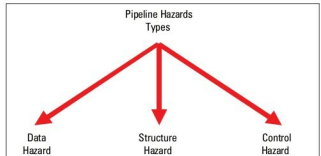


Figure 2: Types of pipeline hazards

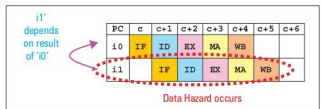


Figure 3: Data hazard

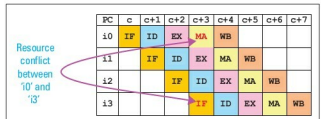


Figure 4: Structure hazard (resource conflict)

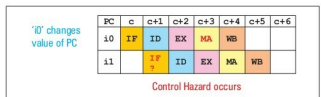


Figure 5: Control hazard

Bypassing

Data hazard can be prevented using a technique known as bypassing (also called 'data forwarding' or 'short-circuiting'). In this technique, the result is moved from the pipeline stage to the stage that requires the result. To explain